

Практическая работа: «Панель администратора с динамическими дашбордами»

Темы: Razor-синтаксис, Layouts, ViewComponents

Общее описание

Создать страницу админ-дашборда с карточками показателей, боковой панелью со сводкой по трендам и многоуровневым layout.

Фокус — на приёмах Razor: выражения, блоки кода, @functions, секции, условная логика, циклы, partial-представления и ViewComponent.

Задание 1. Модели и хранилище

1. Создайте модель DashboardCard:

- ✓ Id (int)
- ✓ Title (string) — заголовок карточки
- ✓ Value (decimal) — значение показателя
- ✓ Trend (enum: Up, Down, Stable) — динамика
- ✓ Unit (string) — единица измерения («руб.», «шт.», «%»)
- ✓ Description (string) — пояснение

2. Создайте интерфейс IDashboardRepository:

- ✓ GetAllCards() — все карточки
- ✓ GetById(int id) — одна карточка
- ✓ GetLatest(int count) — последние по Id

3. Реализуйте InMemoryDashboardRepository со статическим списком из 6 карточек (разные тренды, единицы, значения — включая одно отрицательное). Зарегистрируйте как Singleton в Program.cs.

Задание 2. Многоуровневый Layout

2.1. Базовый layout — Views/Shared/_Layout.cshtml

- HTML-каркас: <!DOCTYPE>, <head>, <body>
- В <title> — @ViewData["Title"]
- Шапка: логотип «Админ-панель» + навигация (ссылки: «Дашборд», «Настройки» — заглушки)
- Подвал: © @DateTime.Now.Year Админ-панель
- @RenderBody() — основное содержимое
- Секция @await RenderSectionAsync("Scripts", required: false)
- Секция @RenderSection("PageHeader", required: false) — кастомный заголовок страницы
- Условная проверка: если IsSectionDefined("PageHeader") — вывести секцию, иначе — дефолтный заголовок «Панель управления»

2.2. Дочерний layout — Views/Shared/_AppLayout.cshtml

- Layout = "_Layout" — вкладывается в базовый
- Двухколоночная разметка: основная область + боковая панель
- В боковой панели — @await Component.InvokeAsync("TrendSummary")
- @RenderBody() — в основную область

2.3. _ViewStart.cshtml и _ViewImports.cshtml

- _ViewStart — Layout = "_Layout" по умолчанию
- _ViewImports — @using для моделей проекта, @addTagHelper

Задание 3. Страница дашборда — Razor-логика

Views/Dashboard/Index.cshtml:

- Layout = "_AppLayout"
- Модель — IEnumerable<DashboardCard>
- ViewData["Title"] = "Дашборд"

Обязательные элементы:

1. Блок @functions с методом:

- ✓ string GetTrendLabel(Trend t) — возвращает текст по тренду: Up → «Рост», Down → «Снижение», Stable → «Стабильно»

2. Блок @{ } в начале:

- ✓ Локальная переменная totalValue — сумма всех Value
- ✓ Локальная переменная cardCount — количество карточек

3. Секция @section PageHeader:

- ✓ Заголовок «Дашборд» + подзаголовок: @cardCount показателей, сумма: @totalValue @ViewBag.Currency
- ✓ В ViewBag.Currency из контроллера передаётся «RUB»

4. Цикл @foreach по карточкам:

- ✓ Заголовок карточки — через @card.Title
- ✓ Значение — через @card.Value @card.Unit
- ✓ Тренд — через вызов @GetTrendLabel(card.Trend)
- ✓ Условие: если card.Value < 0 — вывести бейдж «Отрицательное значение»
- ✓ Условие: если card.Trend == Trend.Up — вывести символ «↑», если Down — «↓», иначе «→»
- ✓ Описание — через @card.Description

5. Счётчик в цикле — перенос строки каждые 3 карточки:

- ✓ Объявите int i = 0; в блоке @{ }
- ✓ Внутри цикла — if (i > 0 && i % 3 == 0) { <div>...</div> } для переноса
- ✓ Увеличение i++ в конце итерации

6. Сводка после цикла:

- ✓ «Всего карточек: @cardCount»
- ✓ «Сумма значений: @totalValue»

Задание 4. Partial-представление карточки

Views/Shared/_DashboardCard.cshtml:

- Модель — DashboardCard (типизированная)
- Содержимое:
 - Заголовок @Model.Title
 - Значение @Model.Value @Model.Unit
 - Тренд через @GetTrendLabel(Model.Trend) (метод из @functions внутри partial)
 - Условие на отрицательное значение
 - Описание @Model.Description
- **Запрещено** использовать ViewBag внутри partial — только типизированная модель

Вызов из Index.cshtml: @await Html.PartialAsync("_DashboardCard", card)

Замените часть цикла в Index.cshtml на вызов partial. Часть карточек выведите напрямую в цикле, часть — через partial, чтобы показать оба подхода.

Задание 5. ViewComponent «Сводка по трендам»

Класс TrendSummaryViewComponent

- В папке ViewComponents/
- Через DI получает IDashboardRepository
- Метод Invoke():
 - Получает все карточки
 - Группирует по Trend
 - Для каждого тренда считает: количество карточек, сумму значений
 - Возвращает View(model) — список сводок

Представление Views/Shared/Components/TrendSummary/Default.cshtml

- Заголовок «Сводка по трендам»
- Цикл по сводкам:
 - Название тренда
 - Количество карточек
 - Сумма значений
- Если данных нет — сообщение «Нет данных»

Вызов

В _AppLayout.cshtml — @await Component.InvokeAsync("TrendSummary")

Задание 6. Работа с ViewData / ViewBag

1. В DashboardController.Index():

- ✓ Передайте список карточек как типизированную модель
- ✓ ViewData["ReportPeriod"] = "Сентябрь 2026"
- ✓ ViewBag.Currency = "RUB"

2. В Index.cshtml:

- ✓ Выведите ReportPeriod из ViewData в подзаголовке
- ✓ Выведите Currency из ViewBag рядом с суммой
- ✓ Добавьте Razor-комментарий @* Данные за период из контроллера *@ над подзаголовком

3. Добавьте HTML-комментарий <!-- Админ-дашборд v1.0 --> в начало страницы.

Структура проекта

```
DashboardAdmin/
├── Controllers/
│   └── DashboardController.cs
├── Models/
│   ├── DashboardCard.cs
│   └── DashboardViewModel.cs
├── Services/
│   ├── IDashboardRepository.cs
│   └── InMemoryDashboardRepository.cs
├── ViewComponents/
│   └── TrendSummaryViewComponent.cs
├── Views/
│   ├── _ViewStart.cshtml
│   ├── _ViewImports.cshtml
│   ├── Shared/
│   │   ├── _Layout.cshtml
│   │   ├── _AppLayout.cshtml
│   │   ├── _DashboardCard.cshtml
│   │   └── Components/
│   │       └── TrendSummary/
│   │           └── Default.cshtml
│   └── Dashboard/
│       └── Index.cshtml
└── Program.cs
```

Дополнительные задания (на «отлично»)

Уровень: Лёгкий

Razor-форматирование и условные бейджи

В `_DashboardCard.cshtml` добавьте метод в `@functions`:

- `string GetValueClass(decimal value)` — возвращает текст:
 - значение больше 0 и меньше 1000 → «Низкое»
 - от 1000 до 10000 → «Среднее»
 - больше 10000 → «Высокое»
 - меньше 0 → «Отрицательное»

Выведите результат вызова рядом со значением карточки. Покажите, как метод вызывается из Razor-разметки.

Что демонстрирует: `@functions`, вызов метода с параметром, условная логика.

Уровень: Средний

Динамический layout через `_ViewStart` + секция с условным рендерингом

1. В `_ViewStart.cshtml` реализуйте выбор layout:

- ✓ Если в `ViewData["Mode"]` передано "Print" — использовать `_PrintLayout` (упрощённый каркас без навигации и подвала)
- ✓ Иначе — `_AppLayout`

2. Создайте `_PrintLayout.cshtml` — вкладывается в `_Layout`, но без шапки и подвала (только `@RenderBody()`).

3. В `DashboardController` добавьте действие `Print()`:

- ✓ Передаёт `ViewData["Mode"] = "Print"`
- ✓ Возвращает тот же список карточек

4. В `_Layout.cshtml` добавьте секцию `@RenderSection("Actions", required: false)`. В `Index.cshtml` определите `@section Actions` с кнопкой «Печать» (ссылка на `/Dashboard/Print`). В `Details.cshtml` (если есть) эту секцию не определяйте — используйте `IsSectionDefined`, чтобы проверить.

Уровень: Сложный

Razor-шаблонный делегат + комбинированная логика

1. В `Index.cshtml` объявите шаблонный делегат для рендеринга карточки:

```
@{
    Func<DashboardCard, Microsoft.AspNetCore.Html.IHtmlContent> renderCard =
        @<div class="card">
            <h4>@item.Title</h4>
            <p>@item.Value @item.Unit — @GetTrendLabel(item.Trend)</p>
            @if (item.Value < 0)
            {
                <span>Отрицательное значение</span>
            }
        </div>;
}
```

2. Используйте этот делегат в цикле вместо `partial`:

```
@foreach (var card in Model)
{
    @renderCard(card)
}
```

3. Добавьте второй делегат `renderSummary`, который принимает `IEnumerable<DashboardCard>` и выводит сводку: количество, сумма, среднее значение.

4. В конце страницы вызовите `@renderSummary(Model)`.

5. В блоке `@functions` добавьте метод `GetAverage(IEnumerable<DashboardCard> cards)`, который используется внутри `renderSummary`.

6. Объясните в комментариях Razor (`@* ... *@`), чем шаблонный делегат отличается от `partial` и когда что лучше использовать.